

Software Development Lifecycle

Sometimes, if you've looked at an application it mentions *certain terms* like; "Waterfall, Modified Waterfall, Agile, Scrum, Software Development Life Cycle"

These, are the **METHODOLOGIES** for making *real world applications*. You don't just code the application in the real world, that's just one part. You have to **plan it, communicate, adapt**.

- **This includes;** Diagramming, getting the real world use of the systems, communicating with the users to **UNDERSTAND** the *how* of the system.

The **ENCOMPASSING** term is Software Development Life Cycle, the 5W's and How of your project's lifecycle, *start to finish*.

When you are working on a project, you'll fall into one (**or more**) of these stages

- **Identify Problem & Get Approval**
 - (Meet with client, see what's wrong, define high level project to fix that, approve budget)
 - **Plan & Monitor Project**
 - (Delegate roles, select project timeline, and **project methodology**)
 - **Discover & Understand Details**
 - (Collect info and understand *in depth* what the customer wants and **how** to implement them, via interviews, review old systems.)
 - **Design System Components**
 - (Technical Diagramming, choosing coding language)
 - **Build/test**
 - (Building the system)
 - **Deploy**
 - (Deploy, and restart the cycle)
-

No matter what stage you are in, or what **METHODOLOGY** you chose; you will have to do quite a lot of analysis, and with it *diagramming and documentation (which is what this lab focuses on)*.

Before we get to that, *first* let's understand the different **METHODOLOGIES**.

Waterfall, this is the most basic approach. It's **STATIC** and **LINEAR**, meaning you *CAN'T* adapt to the changes, or go back to a previous stage, or *be in more than one stage at a time*.

- So in waterfall, if *you get a budget change*, it does not allow you to revise the **system requirements**.

> *The waterfall methodology is only good for **very simple**, and **constant** applications*

Modified Waterfall is a more popular version, it allows **OVERLAPPING** stages (saves time) as well as *allowing you to go backwards* in the case of a flaw.

- *Critical error* in the **BASE** design found in stage four, now you **CAN go back and fix it**.

These two above still have one thing in common, *they attempt to make the **entire system***.

Agile (Scrum) is a philosophy where you take the ideology of being able to go back and between stages, but takes the *entire project in pieces*.

- *Take the idea* of building an entire house; versus building *the kitchen*, then *the bedroom*, then *the closet*, then **eventually the entire house**.

A key thing in **Agile**, in specific Scrum (*which is a way of doing Agile*)

- Is to have **Scrum MEETINGS** (Daily Progress check ins)
- And **SPRINTS**, these are *FIXED* coding and development time.

Sprints focus on *delivering part by part the entire system*, from most important to least important.

- Although Scrum is adaptable, the **time-frame and scope** of a sprint does **NOT** change.
-

That's it for the **METHODOLOGIES**, we focus on the actual analysis (documentations & diagramming) of the SDLC, but it is good to know how the real world works. Next part will cover **SYSTEM ANALYSIS**.